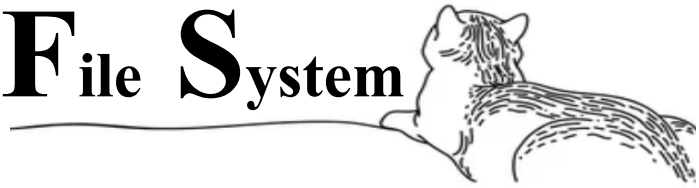
CATS File System



Names: Milo Pesce Ares, Allie Young, Rosa Melton, Abha Deshpande, Patricia Ann Payumo
IDs: MPA 921926821, AY 923129607, RM 921337778, AD 923314454, PAP 923555591
Github Name: milo-pa
Github Link: https://github.com/CSC415-2026-Spring/csc415-filesystem-CATS
Team Name: CATS
Term: Spring 2026
Section: 3

Table of Contents



- CATS File System....1
- Table of Contents....2
- Contributions....3
- HexDump....5
- VCB Struct Description....24
- Free Space Struct Description....25
- Root Directory Description....27
- Team Coordination....28
- Issues Encountered....30

Contributions 

- A table of who worked on which components

Allie	- Conceptual charts/outline - Reformat structs for padding - Add in VCB, extent, directory entry structs - Remove freespace from VCB, replaced with block location - Add root dir length in VCB - VCB function signatures - Implement loadVCB, initVCB, isVCBOurs - getBlocks signature - Implement initFreespace, getBlocks - Remove initRootDir - Rewrote getBlocks to return array of extents
Milo	- Designing directory entry struct - Designing volume control block struct - function signatures for free space - function signatures in fsVCB.h - Dividing group work - Checking for VCB init and instantiating in fsInit.c - File organization, naming, structs and calls - Adding VCB global - Fixing major memory leak that was in loadVCB() - Restructuring usage of VCBGlobal across all files & fsInit.c - Deciding file architecture - Adding Debug helper file and all debug comments - Final comments on all files - Review Hexdumps
Rosa	- formatting and verifying function calls in fsInit.c - Minor edits + additions of error messages for debugging in all files (to be replaced by debug header) - Edit of allocation of buffer in loadVCB for LBAread - Filled in descriptions and resolutions of Issues - Directory && VCB struct Descriptions - Team Coordination section of write-up
Trish	- Format writeup - directory system (fsDir.c && fsDir.h) - createDirectory implementation

	- root directory initialization (“.” && “..”) - scaffolded fsInit - Directory && VCB struct Descriptions
Abha	- HexDump analysis - Added bool check for freespace initialization

HexDump



NOTE: Running lscpu on the VM shows that the byte order on the machine is little endian, meaning the least significant bytes (on the right side) are stored at lower memory addresses. This is why they appear “backwards” in the hexdump.

```
student@student ~/csc415/csc415-assignment-2-bufferandstruct-flimsychip (main*% )++++++++++++ 09:38
└─ lscpu
Architecture:                x86_64
  CPU op-mode(s):            32-bit, 64-bit
  Address sizes:             45 bits physical, 48 bits virtual
  Byte Order:                Little Endian
```

BLOCK 0 (Volume Control Block)

```
000200: 4B 4C 00 00 00 00 00 00  0B 00 00 00 00 00 00 00 | KL**************
000210: 01 00 00 00 00 00 00 00  77 0F DC 8A 00 02 00 00 | ********w*??****
000220: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000230: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000240: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000250: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000260: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000270: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000280: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000290: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0002A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0002B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0002C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0002D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0002E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0002F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

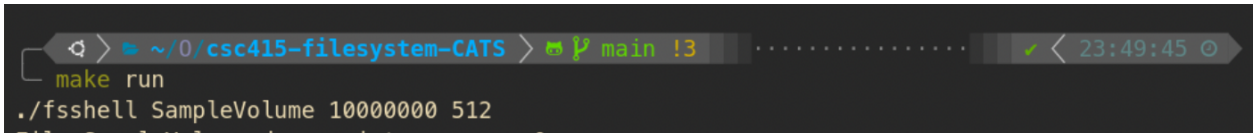
000300: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000310: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000320: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000330: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000340: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000350: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000360: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000370: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000380: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

```
000390: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0003A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0003B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0003C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0003D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0003E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0003F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

Bytes in this color are all unused bytes to fill the remaining block space.

HexDump Analysis:

- ➔ Value: `4B 4C 00 00 00 00 00 00`
- ➔ Address:`000200`
- ➔ Type: `uint64_t`
- ➔ Explanation: This is the blockCount which shows the total number of blocks in the volume. In little endian, this is read as 0x4C4B which is equal to 19531 blocks, which is the same as our volume size of 10000000 bytes / 512 (blockSize), rounded down = 19531 bytes per block. This volume size and block size confirmed by the arguments passed to make run



- ➔ Value: `0B 00 00 00 00 00 00 00`
- ➔ Address: `000208`
- ➔ Type:`uint64_t`
- ➔ Explanation: This is the rootLocation. This is the block number where the root directory lives on our simulated disk. In little endian, `0X0B` translates to 11 which is where the filesystem will look first when navigating through the directory tree. This is the block number where the root director lives, as we confirmed by printing the value and by our analysis of blocks 11 and 12 further down

- ➔ Value:`01 00 00 00 00 00 00 00`
- ➔ Address:`000210`
- ➔ Type:`uint64_t`

- ➔ Explanation:This shows the freeSpaceLocation. In little endian, this is equivalent to `0x01 = 1`, which describes the blockNumber where the free space extent table begins.

- ➔ Value:`77 0F DC 8A`
- ➔ Address:`000218`
- ➔ Type:`uint32_t`
- ➔ Explanation:This shows the signature for our filesystem. The Little-endian for this is: `0x8ADC0F77 = 2329677687` which matches the number we had hardcoded as our system signature — it's all of our initials translated to phone number digits: AD, AY, MP, RM, TP

- ➔ Value:`00 02 00 00`
- ➔ Address:`00021C`
- ➔ Type:`uint32_t`
- ➔ Explanation:This shows the blockSize. In little endian, this should be `0x0200` which corresponds to the value 512, the number of bytes per block.

BLOCK 1

```
000400: 0D 00 00 00 00 00 00 00  3E 4C 00 00 00 00 00 00 | ********>L******
000410: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000420: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000430: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000440: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000450: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000460: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000470: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000480: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000490: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0004A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0004B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0004C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0004D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0004E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0004F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

000500: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000510: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000520: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000530: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000540: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000550: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000560: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000570: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000580: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000590: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0005A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0005B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0005C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0005D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0005E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0005F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

- ➔ Value:`0D 00 00 00 00 00 00 00`
- ➔ Address:`000400`
- ➔ Type:`uint64_t`
- ➔ Explanation:This is the extent[0].location. The starting block number of the first free extent. 0x0D means 11 in decimal, which means that the free space begins at block 11. This is confirmed by the fact that our reserved blocks are 1-10 and root directory takes up two blocks from 11-12, therefore the free space start at 13. These addresses are confirmed by analysis further down

- ➔ Value:`3E 4C 00 00 00 00 00 00`
- ➔ Address:`000408`
- ➔ Type:`uint64_t`
- ➔ Explanation:This is extent[0].length. This shows the number of free blocks to this extent. 0X4C3E would translate to 19518 in decimal which means there are 19518 contiguous blocks of free space ready to use. As we stated above, we have 19531 blocks in our volume. 19531 - 13 = 19518

- ➔ Value:`00 00 00 00 00 00 00 00` (repeated)
- ➔ Address:`000410` onwards
- ➔ Type:`uint64_t` pairs
- ➔ Explanation: These all contain extent[1] through extent[n]. They all contain zeroes showing that there are no additional extents yet. 0 shows that the free slot is yet to be used.

BLOCK 2

```
000600: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000610: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000620: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000630: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000640: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000650: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000660: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000670: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000680: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000690: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0006A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0006B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0006C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0006D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0006E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0006F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

000700: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000710: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000720: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000730: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000740: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000750: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000760: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000770: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000780: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000790: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0007A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0007B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0007C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0007D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0007E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0007F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

BLOCK 3

```
000800: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000810: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000820: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000830: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000840: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000850: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000860: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000870: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000880: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000890: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0008A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0008B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0008C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0008D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0008E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0008F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

000900: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000910: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000920: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000930: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000940: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000950: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000960: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000970: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000980: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000990: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0009A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0009B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0009C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0009D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0009E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0009F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

BLOCK 4

```
000A00: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000A10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000A20: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000A30: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000A40: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000A50: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000A60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000A70: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000A80: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000A90: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000AA0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000AB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000AC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000AD0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000AE0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000AF0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

000B00: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000B10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000B20: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000B30: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000B40: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000B50: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000B60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000B70: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000B80: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000B90: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000BA0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000BB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000BC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000BD0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000BE0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000BF0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

BLOCK 5

```
000C00: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000C10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000C20: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000C30: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000C40: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000C50: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000C60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000C70: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000C80: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000C90: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000CA0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000CB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000CC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000CD0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000CE0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000CF0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

000D00: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000D10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000D20: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000D30: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000D40: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000D50: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000D60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000D70: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000D80: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000D90: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000DA0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000DB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000DC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000DD0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000DE0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000DF0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

BLOCK 6

```
000E00: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E20: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E30: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E40: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E50: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E70: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E80: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E90: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EA0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000ED0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EE0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EF0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

000F00: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F20: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F30: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F40: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F50: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F70: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F80: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F90: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000FA0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000FB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000FC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000FD0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000FE0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000FF0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

BLOCK 7

```
001000: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001010: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001020: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001030: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001040: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001050: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001060: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001070: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001080: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001090: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0010A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0010B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0010C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0010D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0010E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0010F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

001100: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001110: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001120: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001130: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001140: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001150: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001160: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001170: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001180: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001190: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0011A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0011B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0011C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0011D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0011E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0011F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

BLOCK 8

```
001200: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001210: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001220: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001230: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001240: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001250: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001260: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001270: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001280: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001290: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0012A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0012B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0012C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0012D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0012E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0012F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

001300: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001310: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001320: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001330: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001340: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001350: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001360: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001370: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001380: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001390: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0013A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0013B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0013C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0013D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0013E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

```
0013F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

BLOCK 9

```
001400: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001410: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001420: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001430: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001440: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001450: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001460: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001470: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001480: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001490: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0014A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0014B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0014C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0014D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0014E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0014F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

001500: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001510: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001520: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001530: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001540: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001550: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001560: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001570: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001580: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001590: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0015A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0015B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0015C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0015D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0015E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0015F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

BLOCK 10

```
001600: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001610: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001620: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001630: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001640: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001650: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001660: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001670: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001680: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001690: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0016A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0016B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0016C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0016D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0016E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0016F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

001700: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001710: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001720: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001730: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001740: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001750: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001760: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001770: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001780: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001790: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0017A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0017B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0017C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0017D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0017E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0017F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

BLOCK 11 (Root Directory)

```
001800: E0 02 00 00 00 00 00 00  EA 3B D7 69 00 00 00 00 | ?*******?;?i****
001810: EA 3B D7 69 00 00 00 00  EA 3B D7 69 00 00 00 00 | ?;?i****?;?i****
001820: 0B 00 00 00 00 00 00 00  00 02 00 00 00 00 00 00 | ****************
001830: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001840: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001850: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001860: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001870: 01 2E 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | *.**************
001880: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001890: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0018A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0018B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0018C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0018D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0018E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0018F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

001900: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001910: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001920: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001930: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001940: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001950: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001960: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001970: E0 02 00 00 00 00 00 00  EA 3B D7 69 00 00 00 00 | ?*******?;?i****
001980: EA 3B D7 69 00 00 00 00  EA 3B D7 69 00 00 00 00 | ?;?i****?;?i****
001990: 0B 00 00 00 00 00 00 00  02 00 00 00 00 00 00 00 | ****************
0019A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0019B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0019C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0019D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0019E0: 01 2E 2E 00 00 00 00 00  00 00 00 00 00 00 00 00 | *..*************
0019F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

➔ Value：`E0 02 00 00 00 00 00 00`

➔ Address:`001800`

- ➔ Type:`uint64_t`
- ➔ Explanation: This is the size of the directory entry measured in bytes. 0X02E0 in little endian which is the same as 736 bytes. So size = 736 bytes is the total space used by the directory entries. This is because the directory entry was created with the initialEntryCount = 2. Because of the following line of code in fsInit:
  directory_entry *root = createDirectory(2, NULL);

- ➔ Value:`EA 3B D7 69 00 00 00 00`
- ➔ Address:`001808`
- ➔ Type:`time_t`
- ➔ Explanation:This is the creationDate which is of time_t, which typically takes up 8 bytes. To check what date this corresponds to, this is still to be converted to little endian, which turns into `0X69D73BEA` = 1,775,929,322 seconds, since January 1, 1970 which corresponds to approximately April 2026.

- ➔ Value:`EA 3B D7 69 00 00 00 00`
- ➔ Address:`001810`
- ➔ Type:`time_t`
- ➔ Explanation:This is the lastEditDate, the same value as the creationDate. The value is the same, and repeated for lastAccessDate as well since the directory was just created.

- ➔ Value:`EA 3B D7 69 00 00 00 00`
- ➔ Address:`001818`
- ➔ Type:`time_t`
- ➔ Explanation:This is the lastAccessDate, the same value as the creationDate as well.

- ➔ Value:`0B 00 00 00 00 00 00 00`
- ➔ Address:`001820`
- ➔ Type:`uint64_t`
- ➔ Explanation:This shows extent[0].location which is the starting block number for where the root directory data is stored on the disk. 0x0B is 11 in decimal which is indeed the location in which the root directory lives, block 11.

- ➔ Value:`00 02 00 00 00 00 00 00`
- ➔ Address:001828

- ➔ Type:uint64_t
- ➔ Explanation: This shows extent[0].length. 0x0400 in little endian translates to 1024 which is the total size of the root directory data on the disk. 1024 is the total bytes of data allocated on disk for the root directory

- ➔ Value:`00 00 00 00 00 00 00 00`
- ➔ Address:`001830`
- ➔ Type:`uint64_t`
- ➔ Explanation:This shows extent[1] to extent[4] which are the remaining 4 unused exts in the array. The root directory only needs one extent to store its data, the rest of the extents are zeroed out.

- ➔ Value:`01`
- ➔ Address:`001870`
- ➔ Type:`bool`
- ➔ Explanation:This shows the entryType in the directory. 0 corresponds to a file and 1 corresponds to a directory. Since the value is 1, it means that it's a directory.

- ➔ Value:`00 00 00 00 00 00 00 00` (repeated)
- ➔ Address: `001873 - 00196F`
- ➔ Type:
- ➔ Explanation: This is all the unused space in the 255 byte array for the directory entry name.

- ➔ Value: `01`
- ➔ Address: `0019E0`
- ➔ Type: `bool`
- ➔ Explanation: This shows the entryType in the directory. 0 corresponds to a file and 1 corresponds to a directory. Since the value is 1, it means that it's a directory.

- ➔ Value: `2E 2E`
- ➔ Address: `0019E1 - 0019E2`
- ➔ Type: `char`
- ➔ Explanation: This corresponds to the name of the second directory entry. The ASCII for 2E 2E corresponds to "..", which means the current directory.

- ➔ Value: `00`
- ➔ Address: `0019E3`
- ➔ Type: `char`
- ➔ Explanation: This is a null terminator(‘\0’), right after the name of the directory “..”.

The following screenshot has some debug statements used to check whether the values in the hexdump are correct.

```
[DEBUG] fsFreeSpace.c:73: freespaceLocation: 1
[DEBUG] fsFreeSpace.c:75: firstFreeBlock: 11
[DEBUG] fsFreeSpace.c:77: extents[0].location: 11
[DEBUG] fsFreeSpace.c:79: extents[0].length: 19520
[DEBUG] fsFreeSpace.c:81: numExtents: 320
[DEBUG] fsInit.c:72: Free space initialized successfully
[DEBUG] fsDir.c:49: Creating directory with 2 entries, which requires 1024 bytes and 2 blocks
.
[DEBUG] fsInit.c:93: Root directory created successfully
[DEBUG] fsInit.c:104: VCB written to disk successfully. File system initialized
```

BLOCK 12

```
001A00: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001A10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001A20: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001A30: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001A40: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001A50: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001A60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001A70: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001A80: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001A90: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001AA0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001AB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001AC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001AD0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001AE0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001AF0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

001B00: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001B10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001B20: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001B30: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001B40: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001B50: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001B60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001B70: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001B80: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001B90: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001BA0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001BB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001BC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001BD0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001BE0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001BF0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

- ➔ Value: `00 00 00 00 00 00 00 00` (repeated)
- ➔ Address:
- ➔ Type:
- ➔ Explanation: This is padding/free space, not currently occupied.

VCB Struct Description 

The volume control block is the central metadata structure of the file system and is stored at block 0 of our volume. It contains critical information needed to interpret the layout of the volume, functioning as the backbone of the file system by maintaining different core components to ensure that the system can correctly interpret the disk structure, recover its state after restarting, and provide consistency in accessing files and directories.

Our VCB consists of the following 64-bit integer members: blockCount holding the total blocks in our volume, rootLocation holding the location of our root directory, fsLoc holding the location of the start of our freespace extent array (to be initialized with 10 extents). These members are followed by two 32-bit integers: signature, our ‘magic number’ which holds the hex value of our team members’ initials combined (ADAYMPRMTP), and blockSize, the number of bytes per block which is assigned from the passed value through initFileSystem. The sizes of these data types worked out perfectly because the size of our VCB came out to 16 bytes leaving us with no external padding (which we love to see).

During initialization, the VCB is loaded from disk into memory via loadVCB() function. This function allocates memory based on the size of our VCB struct and our static block size, and reads the first block of the volume via LBAread(). The signature member is then read once it is loaded to see if it matches our expected signature. If it does match,the file system assumes that the disk contains valid metadata and skips the formatting process. If it doesn’t, the system initializes the VCB by first setting the block count, size, and signature, followed by initializing its free space and root directory. The free-space global extent array initializes first, from which we store its starting block location in the VCB. The root directory is then created and initialized, again storing its starting block location in VCB. Once all of these are populated, the VCB then is written back to disk using LBAwrite(), ensuring that the file system state is preserved across executions.

Free Space Struct Description 

The free space system is responsible for tracking all unallocated blocks on the volume and efficiently providing space for files and directories. We are using a two-level extent table system to track free space. Each extent consists of two 64-bit integers: a location indicating the starting block and a length indicating how many contiguous blocks are available. Extents will be organized into primary and secondary which are created as needed as the disk becomes fragmented and the number of free extents increases. We are not implementing a tertiary extent table as it is not called for in this project's scope.

The free space system is initialized during file system formatting through the initFreeSpace() function. The starting location of the free space extent array is dynamically calculated based on the size of the VCB and block size to ensure that it is placed immediately after the VCB on disk. We allocate a fixed number of blocks (currently 10) to store the extent table. The number of extents that can be stored is determined by how many extent structures fit within those blocks.

At initialization, the system creates a single large extent representing all available space after the reserved regions (VCB and free space blocks). This extent is stored both in memory (via the global pointer freespaceExtsGlobal) and written to disk using LBAwrite(). All remaining entries in the extent table are initialized to zero and act as sentinels to indicate unused entries. A global flag (freeSpaceInitialized) is set to ensure that allocation functions are not used before initialization.

Block allocation is handled by the getBlocks() function, which implements first-fit allocation. The function scans the extent table for available space and selects extents that satisfy the requested number of blocks and a minimum contiguous allocation size. If a large enough extent is found, that extent is allocated and the corresponding free space extent is updated by adjusting its starting location and reducing its length.

The function returns multiple extents if the requested space cannot be satisfied contiguously. These allocated extents are returned as a dynamically allocated array, terminated by a sentinel extent with zeroed fields. Once allocation is complete, the updated free space extent table is written back to disk to ensure persistence across executions.

If insufficient space is available to satisfy the request, the function returns NULL, indicating allocation failure.

Root Directory Description 

Our directory system is implemented using a directory_entry structure that stores all essential metadata for files and directories. Each entry contains a 64-bit size field, timestamps (creation, last edit, and last access), an array of extents for block locations (initialized with 5 extents), a 1 byte Boolean indicating whether the entry is a file or directory, and a name field of 255 bytes. Unused entries are marked by setting the first character of the name to null terminator ('\0'), telling us which are free entries. Given these, the total size of directory_entry struct includes 24 bytes of external padding.

The root directory is created by calling createDirectory() function from fsDir.c. This function allocates memory for the directory based on the requested number of entries, rounding up to a multiple of the block size to ensure compatibility with disk operations. It then requests contiguous blocks from the free-space system and initializes the directory structure in memory before writing it to disk.

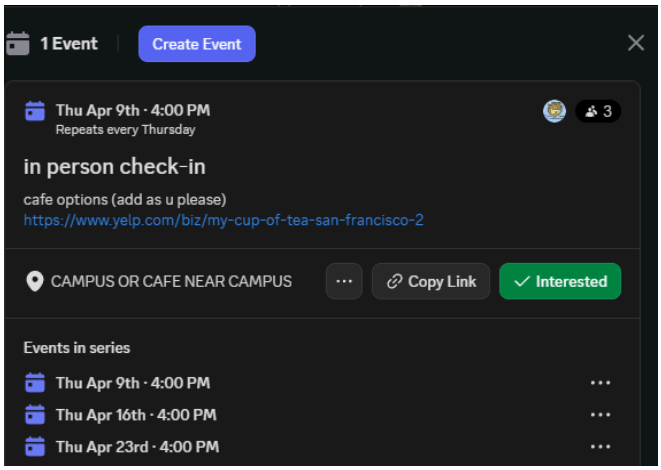
For the root directory specifically, special handling is applied since it has no parent. The required "." entry is initialized to reference the root directory itself, including its metadata and extents. The ".." entry, which normally points to the parent directory, is also set to reference the root directory, ensuring consistency with file system conventions.

After initialization, all remaining directory entries are marked as unused with calloc(), then is written to disk using LBAwrite, and its starting block location is stored in the Volume Control Block (VCB). This allows the file system to reliably locate the root directory during future operations.

The root directory plays a critical role in system initialization, as it serves as the entry point for all file system navigation. During formatting in fsInit.c, the free-space system is first initialized, followed by the creation and persistence of the root directory, establishing the foundation for the entire file system hierarchy.

Team Coordination





How often you met

We met twice a week starting the week of the File System Design deadline.

How did you meet

The first meeting was in person and the other was over a voice call, and in between those calls we messaged frequently over our Discord server to check in with our understanding of our progress as we went.

How did you divide up tasks

a) File System Design Assignment 1 - Directory entry design

Creating the directory entry struct was done in one session and took a couple hours to complete together. This assignment did not require delegation as it required everyone to be on the same page to put together this foundational struct. The core of the struct was already completed given the previous lectures on what a directory entry should have, and we agreed to add dates for creation and last edited/accessed for the user's convenience. While not everyone could be present for the meet up in the library, we made up for it by bringing everyone up to speed with the reasons behind each variable and data type chosen for our directory entry.

b) File System Design Assignment 2 - VCB and free space prototypes

We divided this assignment into three parts: defining the struct for VCB, outlining the free space design and how it will be tracked, and creating the function signatures for our free space. This work was done together over a voice call on a shared doc so we could check in with each other's work and make sure the pieces were fitting together. VCB was outlined by Allie, Rosa defined extent and extent_table structs (deprecated the latter), Trish and Abha worked on the function prototypes for our extents and VCB, and Milo bounced between helping Allie and making sure the rest followed this schema

c) File System Milestone 1 - Implementing Assignments 1 and 2

This milestone took about 2 weeks to really get pieced together, starting with a meeting at a cafe (see the Discord calendar screenshot above) Here, since we had the File System Design assignment to work off of, the first step was to delegate who would actually implement our VCB struct and what functions would be needed to interact with this struct. In this meeting, Allie

handled practically all of the header and C files for this with Milo “back-seat coding”. Abha started implementing the free space header and functions, Trish set up our CATS-tastic write-up, and Rosa scaffolded the initialization function for our file system. It took us a while to figure out what was going on, even with the Steps document, but that helped immensely in divvying up the work. After this meeting, we agreed to communicate often in Discord before attempting to work on a file or pushing to Git. From there, since Allie and Milo worked on the central part of the milestone, they delegated various helper functions and methods to the others to finish implementations for fsInit().

Issues Encountered 

Issue # 1

- ➔ **Problem:** had to remove root dir length in VCB, can just read 1 block
- ➔ **Description:** we intended to store the length of the root directory within our VCB so we could call LBAread() with the length as the first parameter, but didn't know how to properly get this length.
- ➔ **Resolution:** Our workaround was to use at least 1 block to feed into LBAread()
  (Issue # 11 ties into this as well)

Issue # 2

- ➔ **Problem:** getBlocks only searches for contiguous space when we want discontiguous extents for files
- ➔ **Description:** A discussion frequently brought up but have not encountered an issue yet with this design so TBD…
- ➔ **Resolution:** Based on feedback in office hours, we changed getBlocks() to return an array of extents instead and added a second parameter for a minimum number of contiguous blocks to be used only when creating directories since we know we want those stored contiguously.

Issue # 2

- ➔ **Problem:** Freespace extents array was inside VCB
- ➔ **Description:** This prevented us from accessing getBlocks() during runtime, which meant we would have to load from disk every time we wanted to call it.
- ➔ **Resolution:** Had to be moved out; storing the starting block of free space location is satisfactory for the rest of our implementations.

Issue # 3

- ➔ **Problem:** Had to pass volume control block to everything
- ➔ **Description:** What the problem says; had a repeated, unnecessary parameter that could be improved.
- ➔ **Resolution:** We are using a global instance of the volume control block so it can be accessed across many files and functions and because there is only one per volume

Issue # 4

- ➔ **Problem:** Had freespace extents global when it did not need to be
- ➔ **Description:** Turned out to be unnecessary implementation. Simple removal

- ➔ **Resolution:** Removed the extern declaration. Freespace extents never need to be accessed by external functions since we have getBlocks

Issue # 5

- ➔ **Problem:** Currently not supporting secondary/tertiary extent tables
- ➔ **Description:** We are not accounting for extents greater than the 10 initialized with the file system, and we will be in trouble if we do not handle this as we write more and more to our volume…
- ➔ **Resolution:** Since this milestone is only focused on initializing everything, these TODOs will be pushed as we fill our volume in the next milestone. Office hours feedback - we will not need tertiary extent tables at all given the scope of this project.

Issue # 6

- ➔ **Problem:** Entire file deleted
- ➔ **Description:** N/A
- ➔ **Resolution:** `git revert`

Issue # 7

- ➔ **Problem:** Memory leak that was incredibly difficult to track
- ➔ **Description:** We were trying to populate fsInit.c with all the completed and compiling files that we had finished. However, we started experiencing inexplicable, intermittent errors. Print statements wouldn't run in order, it changed from machine to machine, and seemingly triggered randomly. Classic C stuff that everyone loves
- ➔ **Resolution:** A buffer that was not evenly divisible by block size was being passed to LBARead(). We think this was the root cause, but with intermittent memory problems you can never be too sure. This led us to include the file and function names in all error messages to help with future tracking.

Issue # 8

- ➔ **Problem:** handling "." and ".." for root directory
- ➔ **Description:** Special case since ".." should point to the parent directory, but for the root directory, there's technically no parent.
- ➔ **Resolution:** In createDirectory, if the parent pointer is NULL, the new directory is treated as the root and ".." is set to point back to itself.

Issue # 9

- ➔ **Problem:** safely initializing unused directories
- ➔ **Description:** if directory memory isn't cleared first, unused entries may contain garbage values which may cause issues when checking if an entry is free or used

- ➔ **Resolution:** decided to replace malloc+memset with calloc() instead to ensure the unused entries are empty (less calls the better; calloc() appropriate for this use);

Issue # 10

- ➔ **Problem:** initFreeSpace() and initVCB() relied on the other to already be initialized and could not resolve without hardcoding in these function calls…
- ➔ **Description:** initVCB required free space location to already be set in initFreeSpace, yet initFreeSpace relied on an already instantiated member of VBC.
- ➔ **Resolution:** Remove initVCB() and assign VCB's members as we call its respective initialization functions (ie createDir() for root, initFreeSpace()

Issue # 11

- ➔ **Problem:** VCB's buffer malloc'd with less than blockSize causing LBAread() issues
- ➔ **Description:** We didn't want to malloc an entire block to be occupied by only 32 bytes for VCB and we are stubborn
- ➔ **Resolution:** There is no way around this we need a block for LBAread to work so we applied the 'blocks needed' formula provided from lecture and malloc'd that to VCB
  "It's okay to waste a little bit."

Issue # 12

- ➔ **Problem:** Making header compiled debug print function
- ➔ **Description:** The file system shouldn't print messages, but we need print statements for debugging. After going to office hours and asking about this, I was told I could add a debug.h file that used macros to define a function that could be called for printing debug statements. However, Bierman warned me that it could be a little tricky, particularly with creating a function that takes in an indeterminate amount of params.
- ➔ **Resolution:** I looked online first for implementation examples on passing multiple parameters to a function and debug macro functions. Here are some of the links that I found helpful:
  - ◆ https://ilostmynotes.blogspot.com/2011/04/ifdef-debug-print-statements-in-c.html
  - ◆ https://sqlpey.com/c/c-cpp-debugging-macros/

  I then added the header file to the makefile and added a simple command to simplify calling it